Universidad Galileo	**Fecha 17/10/2026**
Facultad: FISSIC	**Alumno: Pablo Urbina**
Curso: Microprocesadores	**Carnet: 24001508**
Sección: A	**Hora de Laboratorio: 1:00 PM**
Auxiliar: **Brandon Eduardo Mendoza Espinoza**	**Día de Laboratorio: Lunes**

Laboratorio #8 ADC

Objetivos:

Aprender a utilizar el convertidor Analógico – Digital del microcontrolador, utilizando VS Code.

Resumen:

Se armo un circuito sencillo en el cual se utilizaba un potenciómetro para controlar unos LEDs, donde por medio de la tarjeta, se hizo un programa en ensamblador para configurar el ADC, hacer una conversión y usar el dato convertido para controlar los cuatro LEDs en base a la tabla proveída.

Recorrido	LED1	LED3	LED3	LED4
0 a 1/5	Apagado	Apagado	Apagado	Apagado
1/5 a 2/5	Apagado	Apagado	Apagado	Encendido
2/5 a 3/5	Apagado	Apagado	Encendido	Encendido
3/5 a 4/5	Apagado	Encendido	Encendido	Encendido
4/5 a 1	Encendido	Encendido	Encendido	Encendido

Teoría:

Un convertidor analógico-digital (ADC) permite que un microcontrolador procese señales cuyo voltaje varía de forma continua, el microcontrolador STM32G431 dispone de dos ADC de 12 bits.

La conversión por aproximaciones sucesivas determina un valor digital mediante comparaciones progresivas de la señal de entrada con valores derivados de una referencia. La resolución establece cuántos resultados distintos puede entregar el ADC: con 10 bits existen 2^{10} niveles, numerados del 0 al 1023. Con una referencia de 3.3V, cada nivel representa $\frac{3.3V}{1024} = 3.223\,mV$, la calidad de la medición depende de factores como el tiempo de muestreo y la estabilidad del voltaje de referencia.

El primer paso es habilitar el reloj de los pines que se utilizaran, configurando el pin correspondiente en modo analógico (11), luego, se selecciona el canal del ADC, establece parámetros como la resolución y el tiempo de muestreo, habilita el convertidor y espera a que este listo. En esta práctica, se inicia una conversión por software, se espera a que termine y se obtiene el resultado para tomar una decisión, la cual es cuales LEDs se van a encender.

Datos Prácticos:

Laboratorio funcionando:

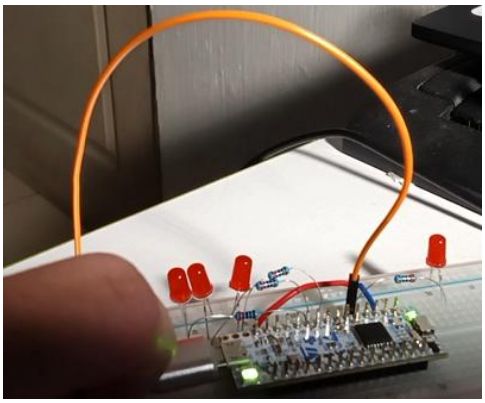
https://drive.google.com/file/d/1Q_SXfQrBexCVxxn4q46kv0VPoHcKaVnk/view?usp=sharing

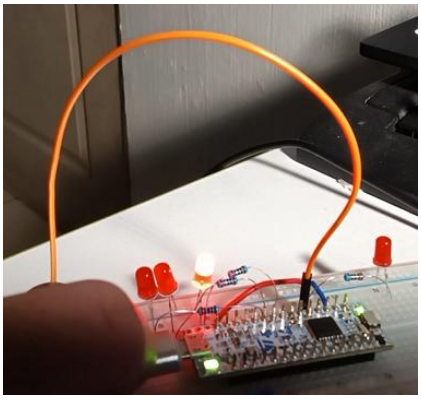
Laboratorio (Paso 5):

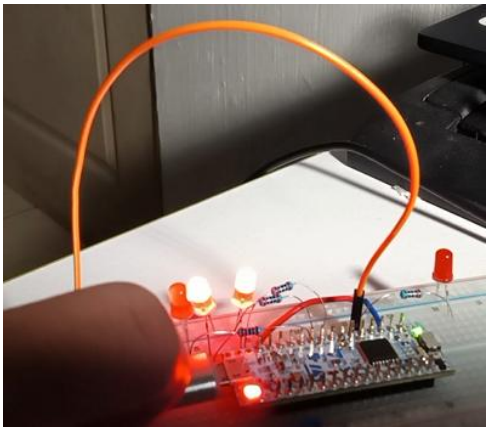
https://drive.google.com/file/d/1TjDjlkl3NySYgxdWHA9xhb75z6EDKhoY/view?usp=sharing

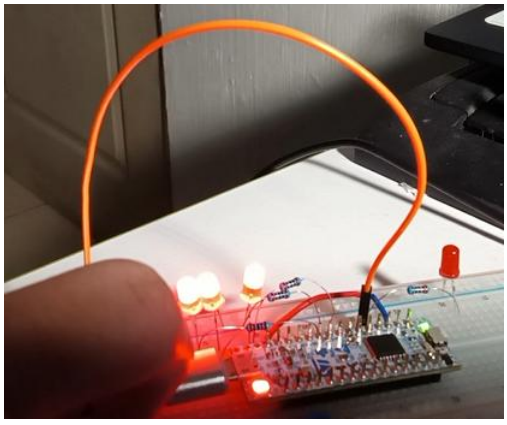
Pines usados: PA1 para el potenciómetro, PA0, PA6, PA7, PA5 para los LEDs

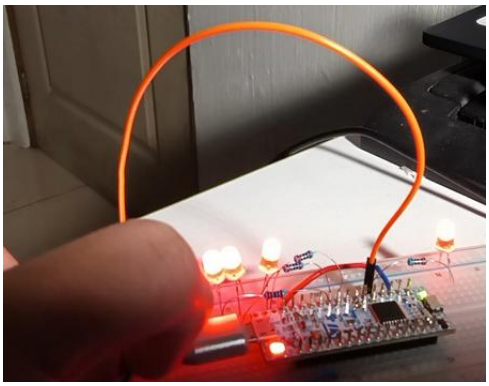
Valor ADC	LED encendido
0-204	Ninguno
205-409	PA5
410-614	PA5, PA6
615-819	PA5, PA6, PA7
820-1023	PA5, PA6, PA7, PA0











Explicación del código (Partes importantes)

```
adc_config
    push {lr}
    ; Activar reloj de los pines
    ldr r0, =0x4002104C ;RCC_AHB2ENR
    ldr r1, [r0]
    ldr r2, =0x00002001
    orr r1, r2
    str r1, [r0]
```

Habilitar relojes de GPIOA y ADC12

```
; PA1 en modo analogico
ldr r0, =0x48000000 ;GPIOA
ldr r1, [r0] ;MODER
orr r1, #0x0C
str r1, [r0]
ldr r1, [r0, #0x0C] ;PUPDR
bic r1, #0x0C
str r1, [r0, #0x0C]
```

Colocar los bits de 3:2 de PA1 en modo analógico (11) y eliminar las resistencias internas pull-up y pull-down del pin

```
ldr r0, =0x50000100
ldr r1, =0x00001008
str r1, [r0, #0x0C]
movs r1, #0x80
str r1, [r0, #0x30]
ldr r1, =0x000001C0
str r1, [r0, #0x14]
ldr r2, =0x50000308
ldr r1, =0x00030000
str r1, [r2]
```

Configurar ADC2 a 10 bits, seleccionando el canal 2, asociado a PA1 y estableciendo el tiempo de muestreo, configurando el reloj común de los ADC

```
    ; Salir de deep power down y activar regulador de ADC
    ldr r1, [r0, #0x08]
    ldr r2, =0x20000000
    bic r1, r2
    ldr r2, =0x10000000
    orr r1, r2
    str r1, [r0, #0x08]
    ldr r1, =2000

adc_wait
    subs r1, #1
    cmp r1, #0
    bgt adc_wait

    ldr r1, [r0, #0x08]
    orr r1, #1
    str r1, [r0, #0x08]

adc_ready
    ldr r1, [r0] ;ISR
    and r1, #1
    cmp r1, #0
    beq adc_ready
    pop {pc}
```

Se desactiva el Deep-power-down, se activa el regulador interno y se espera a que se estabilice, luego se habilita ADC2. Se hace el ciclo adc_ready para impedir que se continue hasta que ADC ya esté funcionando.

```
led_config
    push {lr}
    ldr r0, =0x48000000 ; GPIOA
    ldr r1, [r0]
    ldr r2, =0x0000FC03
    bic r1, r2
    ldr r2, =0x00005401
    orr r1, r2
    str r1, [r0]
    ldr r1, [r0, #0x0C]
    ldr r2, =0x0000FC0F
    bic r1, r2
    str r1, [r0, #0x0C]

    ldr r0, =0x48000018
    ldr r1, =0x00E10000
    str r1, [r0]

    pop {pc}
```

Configurar PA0, PA5, PA6 y PA7 como salidas, se apagan los LEDs con 0x00E10000 al iniciar el programa.

```
adc_read
    push {lr}
    ldr r1, =0x50000100
    ldr r2, [r1, #0x08]
    orr r2, #4
    str r2, [r1, #0x08]

adc_wait_eoc
    ldr r2, [r1]
    and r2, #4
    cmp r2, #0
    beq adc_wait_eoc
    ldr r0, [r1, #0x40]
    pop {pc}
```

Se inicia una conversión, luego en adc_wait_eoc se espera a que la conversión termine, se obtiene el resultado y se guarda en r0, este resultado se usa en led_write.

```
led_write
    push {lr}
    movs r1, #0
    ldr r2, =205
    cmp r0, r2
    blt led_output
    orr r1, #0x20
    ldr r2, =410
    cmp r0, r2
    blt led_output
    orr r1, #0x40
    ldr r2, =615
    cmp r0, r2
    blt led_output
    orr r1, #0x80
    ldr r2, =820
    cmp r0, r2
    blt led_output
    orr r1, #0x01

led_output
    ldr r2, =0x48000018
    ldr r3, =0x00E10000
    str r3, [r2]
    str r1, [r2]

    pop {pc}


    end
```

R0 obtiene la lectura de 0 a 1023, r1 va acumulando los bits de los LEDs que deben encenderse, la primera escritura de led_output apaga los 4 LED y la segunda enciende los LEDs seleccionados.

Para el paso 5, se comentan las dos escrituras finales de led_output, por lo que al no escribir el nuevo estado, los LED permanecen apagados incluso si se mueve el potenciometro

Cálculos Teóricos:

Limites del ADC en 10 bits:

$$2^{10} = 1024$$

$$D_{\max} = 2^{10} - 1 = 1023$$

Hay 5 estados dependiendo del estado del potenciómetro.

$$\frac{1024}{5} = 204.8$$

ADC solo entrega números entero, por lo que se redondea hacia arriba para obtener los valores que se comparan en el programa para saber que LEDs deben encender según el rango en el que se este, es decir, cada limite aumenta en 205 respecto al anterior, por lo tanto, rangos para cada estado son:

$$0 - 204, 205 - 409, 410 - 614, 615 - 819, 820 - 1023$$

Conclusiones:

El cambio de estado de los LEDs al girar el potenciómetro demostró que ADC2 es capaz de obtener las lecturas de señal del pin correspondiente y que un programa es capaz de utilizarlas para controlar las salidas de GPIO, lo cual se vio reflejado en los limites marcados, también, al comentar las escrituras en led_output se demostró que una lectura de ADC no cambia por si sola las salidas, si no que el cambio ocurre cuando el programa escribe el nuevo estado en los GPIO.

Bibliografía:

[1] STMicroelectronics, “NUCLEO-G431KB.” [En línea].

Disponible: https://www.st.com/en/evaluation-tools/nucleo-g431kb.html.

[2] STMicroelectronics, “STM32G431KB.” [En línea].

Disponible: https://www.st.com/en/microcontrollers-microprocessors/stm32g431kb.html.

[3] STMicroelectronics, “Getting started with ADC,” STM32 MCU Wiki. [En línea].

Disponible: https://wiki.st.com/stm32mcu/wiki/Getting_started_with_ADC.

[4] STMicroelectronics, “ADC measurements recommendations,” STM32 MCU Wiki. [En línea]. Disponible: https://wiki.st.com/stm32mcu/wiki/ADC_measurements_recommendations.